STRAX Autonomous QA Platform – Final Enterprise Architecture

1. Overview & Guiding Philosophy

Goal: Build a fully automated, AI-powered SaaS QA platform that can:

- test frontend (UI/UX/responsive)
- test backend APIs
- scan for security vulnerabilities
- execute complete test flows
- generate screenshot/video proof
- provide AI-driven root cause analysis and suggestions
- in future, enable self-healing tests

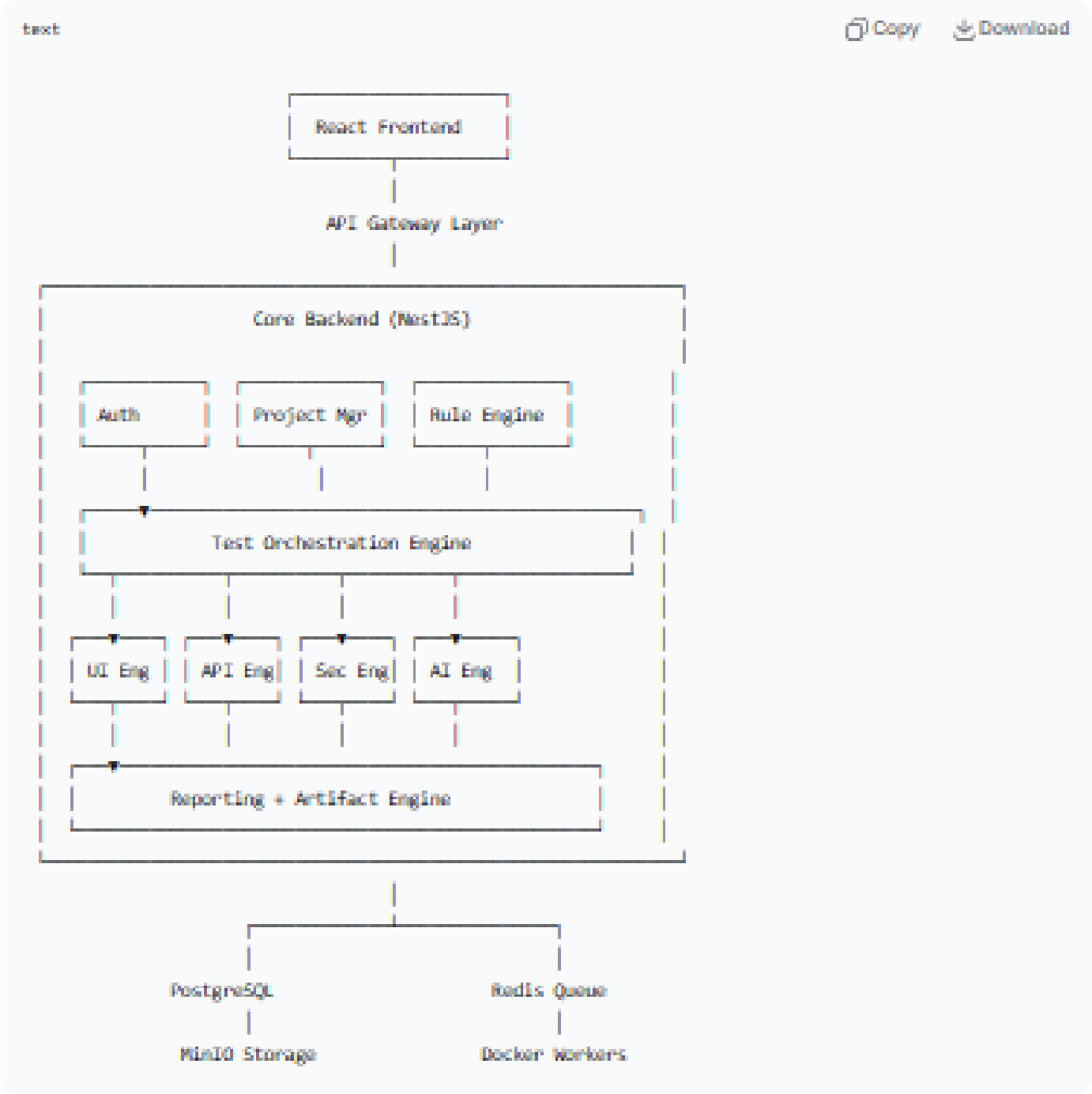
Start with Modular Monolith, Scale to Distributed Workers

Why this approach?

- Faster initial development
- Lower complexity for debugging
- Reduced infrastructure cost
- SaaS-ready from day one
- Easy to scale later when load demands it

The system is built as a **modular monolith** first (all core modules in one deployable unit), then gradually evolves into a **distributed worker architecture** with Redis queues and Docker workers.

2. High-Level System Architecture



- **API Gateway** handles authentication, rate limiting, routing.
- **Core Backend** (NestJS) is the modular monolith containing all business logic.
- **PostgreSQL** stores all relational data (users, projects, tests, results).
- **Redis** handles job queues (BullMQ), caching, sessions, and real-time status.
- **MinIO** (S3-compatible) stores screenshots, videos, logs, reports.
- **Docker Workers** are spawned to run isolated test jobs (Playwright, API tests, security scans).

3. Detailed Module Breakdown

3.1 Auth & Organization Module

Purpose: Multi-tenant SaaS with workspace isolation.

Features:

- Login / signup with JWT
- Organization & workspace management
- Role-based access control (RBAC)
- API key generation & management
- Audit logs
- Session management with Redis

Database Tables:

- organizations
- users
- roles
- permissions
- audit_logs
- api_keys

Free Tools Used:

Purpose	**Tool**
Authentication	JWT
Password Hash	bcrypt
RBAC	CASL
Session Store	Redis

3.2 Project Builder Engine

Purpose: Connect and manage Git repositories, automatically detect frameworks, and prepare the project for testing.

Responsibilities:

- Git repo clone & sync (HTTPS/SSH)
- Branch selection
- PAT / SSH key storage (encrypted)
- Environment variable injection
- Automatic framework detection
- Repository zip extraction

Framework Detection Logic:

Framework	Detection Hint
React	package.json (react)
Laravel	artisan file
Django	manage.py
Spring Boot	pom.xml
Vue	vite.config
Angular	angular.json

Free Tools:

Purpose	Tool
Git operations	simple-git
GitHub API	Octokit
GitLab API	GitLab SDK
Zip extraction	adm-zip

3.3 Rule Engine (Most Critical)

Purpose: Define *what* and *how* to test. All testing standards are encoded as JSON rules.

Architecture:

text

Rule Categories

```
├── UI Rules (colors, spacing, fonts)
├── UX Rules (loaders, toasts, transitions)
├── Security Rules (headers, CORS, tokens)
├── Validation Rules (form fields, error messages)
├── API Rules (status codes, schemas, response times)
└── Accessibility Rules (contrast, ARIA labels)
```

Example Rule Format:

```json
{
  "type": "button",
  "selector": ".primary-btn",
  "rules": {
    "background": "#FF6A00",
    "border-radius": "12px",
    "font-size": "16px"
  }
}
```

Free Tools:

Purpose	Tool
Validation	Zod
Rule parsing	AJV (JSON Schema)
Storage	Native JSON in PostgreSQL

3.4 Code Analyzer Engine

Purpose: Statically analyze source code to extract metadata—pages, routes, APIs, components, forms, styles—without running the app.

Responsibilities:

- Parse frontend code (React, Vue, etc.)
- Detect backend routes and controllers
- Extract components, forms, and their attributes
- Identify CSS classes and style rules
- Map API endpoints used in frontend code

Analysis Flow:

```
text
```

```
Repo → Parser → AST Analysis → Metadata Extraction → Section Generation
```

Free Tools:

Purpose	Tool
JS/TS AST	Babel Parser / ts-morph
CSS Parsing	PostCSS
HTML Parsing	Cheerio
PHP Parsing	php-parser
Python Parsing	Python AST

Detection examples:

- Frontend: axios.get('/api/users') → maps to endpoint
- Backend: router.get('/users', handler) → extracts route definition

Output Metadata:

```
json
```

```
{
  "page": "Dashboard",
  "apis": [
```

```
  { "method": "GET", "path": "/api/stats" },
  { "method": "GET", "path": "/api/users" }
 ]
}
```

3.5 Section Generator Engine

Purpose: Automatically build a hierarchical testable structure of the application (pages, sections, components).

Example Output:

```
text
Dashboard
 ├── Sidebar
 ├── Header
 ├── Charts
 │    ├── Revenue Chart
 │    └── User Growth Chart
 └── User Table
```

How it works:

- Uses route definitions (React Router, Vue Router) to discover pages
- Parses JSX/HTML to identify structural components
- Groups components into logical sections

Free Tools:

Purpose	**Tool**
Route detection	React Router parser
File scanning	fast-glob
JSX parsing	Babel

3.6 Test Task Engine

Purpose: Define runnable test workflows (suites) that group individual test cases.

Example Task Definition:

```
text

Task: Smoke Test

Includes:

 - Login flow

 - Dashboard rendering

 - Critical API responses

 - Basic security headers
```

Implementation:

- Internal module that translates rules + sections into executable jobs
- Uses BullMQ for scheduling and retries

3.7 Test Orchestration Engine (Brain of the System)

Purpose: Manage the entire execution lifecycle—queuing, worker allocation, retries, scheduling, and artifact collection.

Responsibilities:

- Queue management (BullMQ)
- Worker allocation (Docker containers)
- Retry logic (exponential backoff)
- Scheduling (cron-based recurring tests)
- Parallel test execution
- Artifact collection & aggregation

Architecture:

```
text

Test Task → Queue (Redis) → Worker (Docker) → Testing Engine → Results
```

Free Tools:

Purpose	Tool
Job Queue	BullMQ
Scheduling	node-cron
Containerisation	Docker
Parallel Execution	worker_threads

3.8 UI Testing Engine

Final Tool: Playwright

Why Playwright?

- Completely free & open source
- Modern, fast, and reliable
- Multi-browser (Chromium, Firefox, WebKit)
- Built-in screenshot, video, and trace recording
- Mobile device emulation

Features covered:

- **Functional:** click, type, navigation, form fill
- **UI validation:** fonts, colors, spacing (via computed styles)
- **Responsive:** mobile, tablet, desktop viewports
- **UX checks:** loaders, toast messages, popups, animations

Custom Wrapper: To make tests more resilient, we create smart wrappers like:

- smartClick(selector) – waits for element to be stable
- smartType(selector, text) – with built-in delay and validation
- smartNavigate(url) – waits for page fully loaded

3.9 Screenshot & Proof Engine

Purpose: Capture visual evidence of test execution—before/after screenshots, failure highlights, and full-page videos.

Responsibilities:

- Before-action screenshot
- After-action screenshot

- Failure screenshot with highlighted issue
- Video recording of entire test session
- Visual diff between baseline and current run

Free Tools:

Purpose	**Tool**
Screenshots	Playwright
Image Diff	pixelmatch
Video	Playwright
Image Processing	sharp

Storage Structure:

```
text

/artifacts

 /project-id

  /run-id

   /screenshots

   /videos

   /diffs
```

3.10 API Testing Engine

Purpose: Validate backend APIs against expectations (status, schema, auth, performance).

Responsibilities:

- Status code validation
- JSON schema / contract validation
- Authentication & authorization checks (valid, expired, missing tokens)
- Response time thresholds
- Payload structure verification

Free Tools:

Purpose	Tool
HTTP Client	Axios
Schema Validation	AJV
Contract Testing	Pact
Collection Runs	Newman (Postman collections)

3.11 API Extraction & Mapping Engine (Signature Feature)

Purpose: Automatically discover the map between frontend pages and the backend APIs they consume.

Output Example:

```
text

Login Page
  └── POST /api/login

Dashboard
  ├── GET /api/stats
  └── GET /api/users
```

Detection Sources:

Source	Pattern
Axios	axios.get('/api/...')
Fetch	fetch('/api/...')
Custom services	service wrappers

Free Tools:

Purpose	Tool
AST Parsing	Babel
Dependency Graph	Madge

3.12 Security Testing Engine

Purpose: Execute automated security scans to catch vulnerabilities early.

Responsibilities:

- SQL Injection (SQLi)
- Cross-Site Scripting (XSS)
- Authentication bypass attempts
- Rate limiting tests
- CSRF protection checks
- Server-Side Request Forgery (SSRF)
- CORS misconfigurations
- Exposed secrets / API keys

Free Tools:

Purpose	Tool
DAST (Dynamic)	**OWASP ZAP**
Vulnerability Scan	**Nuclei**
Dependency Scan	**OWASP Dependency Check**
Secrets Detection	**Gitleaks**

3.13 Performance Engine

Purpose: Validate system behaviour under load and identify bottlenecks.

Responsibilities:

- Load testing (gradual ramp-up)

- Stress testing (breaking point)
- Concurrency simulation
- Bottleneck detection (slow queries, CPU, memory)

Free Tools:

Purpose	**Tool**
Load Testing	k6
Stress Testing	Artillery
Metrics Collection	Prometheus

3.14 Reporting Engine

Purpose: Aggregate all test results into beautiful, actionable reports.

Responsibilities:

- Combine test outcomes, logs, screenshots, and videos
- Generate PDF and HTML reports
- Executive summary for management
- Technical drill-down for developers
- Security-specific reports for audit

Free Tools:

Purpose	**Tool**
PDF Generation	Puppeteer
Charts	Recharts
HTML Reports	EJS
Markdown Reports	markdown-pdf

Report Types & Audience:

Type	Audience
Executive	Management
Technical	Developers
Security	Security team

3.15 AI Validation Engine

Purpose: Initially acts as a smart assistant; later becomes core to self-healing.

AI Responsibilities:

- Detect **missed bugs** by analysing patterns
- **Root cause analysis** of failures
- Suggest **test flow optimisations**
- Generate new test cases from production traffic patterns
- Self-healing – auto-update selectors when UI changes

Low-Cost AI Stack:

Layer	Tool
LLM Router	OpenRouter
Primary Model	DeepSeek / Gemini Flash
Fallback	Ollama (local)
Embeddings	sentence-transformers
Vector DB	pgvector (PostgreSQL)

Why this stack?

- Extremely cheap compared to OpenAI
- Works offline with Ollama when needed
- Vector similarity search using pgvector keeps everything in one DB

3.16 Artifact Storage Engine

Purpose: Securely store and serve all test artefacts.

What is stored:

- Screenshots (before/after/failure)
- Video recordings
- Raw and parsed logs
- PDF/HTML reports

Free Tools:

Purpose	**Tool**
S3-compatible	MinIO
Compression	sharp
CDN (opt.)	Cloudflare

3.17 Observability Engine

Purpose: Monitor the health and performance of the platform itself.

What is monitored:

- Worker health (crashes, restarts)
- Memory & CPU usage
- Job queue lengths and throughput
- Error logs from all engines

Free Tools:

Purpose	**Tool**
Log Aggregation	Loki
Metrics	Prometheus
Dashboards	Grafana

4. Database Design

PostgreSQL (Relational Core)

- organizations
- users
- roles / permissions
- projects
- repositories
- rules (JSONB)
- test_tasks
- test_runs
- test_results
- api_keys
- audit_logs

MongoDB (Optional – Heavy Logging)

- playwright_logs
- security_scan_logs
- console_logs

Redis (Cache & Queue)

- BullMQ queues
- Session store
- Real-time status updates
- Rate limiting counters

Note: Starting with PostgreSQL + Redis is sufficient. MongoDB can be added later if log volumes become extremely large.

5. Deployment & Scaling Strategy

Phase 1 – MVP Launch

- Single VPS
- Docker Compose (NestJS, PostgreSQL, Redis, MinIO, Workers)
- All workers run on the same machine

Phase 2 – Worker Scale-out

- Dedicated worker nodes
- Redis Queue becomes the central job dispatcher
- NGINX as reverse proxy and static file server

Phase 3 – Full Orchestration

- Kubernetes cluster
- Horizontal Pod Autoscaler for workers
- Distributed testing across regions
- Managed database services (if required)

6. MVP Phases (What to Build & When)

🔴 Phase 1 – Core (Must Build First)

- Authentication & multi-tenancy
- Project builder (Git integration)
- Repo sync & basic framework detection
- Playwright UI engine
- API testing engine
- Screenshot & proof engine
- Basic reporting dashboard

🟡 Phase 2 – Intelligent Testing

- Rule engine (JSON-based validation)
- Section generator (automatic page hierarchy)
- API extraction & mapping
- Queue-based workers (BullMQ)

🟢 Phase 3 – Advanced & AI

- Security engine (ZAP, Nuclei)
- Performance testing (k6)
- AI validation (root cause, suggestions)
- Self-healing capabilities
- Visual AI (layout-aware diffs)

7. Final Verdict & Honest Assessment

The architecture is:

- **Scalable** – the modular monolith → distributed worker path is proven
- **SaaS-ready** – multi-tenant, role-based, API-key driven
- **Enterprise compatible** – audit logs, observability, security scans
- **AI-extensible** – low-cost AI integration without vendor lock-in
- **Cost-optimised** – overwhelmingly uses free & open-source tools

Honest Assessment:
If executed with discipline, this product has the potential to become one of India's strongest AI-driven QA SaaS platforms. The vision is complete, the architecture is sound, and the technology choices are pragmatic.

Challenges to anticipate:

- The **rule engine** must be exceptionally flexible yet simple to use—this will require iterative refinement.
- **Code analysis** across multiple frameworks (React, Vue, Laravel, Django) is non-trivial; start with JavaScript/TypeScript only and expand.
- **AI self-healing** is a long-term feature; do not over-promise it initially.
- **Worker orchestration** at scale will need careful monitoring and retry logic to avoid flakiness.

Bottom line: This is a well-thought-out, modern, and ambitious architecture. Build it step by step, validate with real users, and you'll have a product that genuinely solves a massive gap in the QA automation market. 🚀